

Intelligent Cloud Document Analyst

n8n AI Engineering Project — Powered by Google Gemini API

What makes this project different?

This upgraded project replaces local Ollama models with the Google Gemini API — a production-grade, cloud-based LLM. Students will work with real API authentication, multi-modal AI (text + images), streaming responses, and build a full-stack intelligent document pipeline used in real enterprise workflows.

1. Project Summary

Students will design and build a complete cloud-connected automation pipeline that intelligently processes documents using n8n and the Google Gemini API. The system watches a local or cloud-synced folder for incoming files, extracts their content, and routes them through a multi-stage Gemini AI pipeline for summarization, classification, sentiment analysis, and entity extraction.

Unlike purely local setups, this project requires students to understand real-world API usage patterns: authentication via API keys, handling rate limits, parsing structured JSON responses, and connecting AI output to Google Sheets as a cloud results database and Gmail for real-time email notifications.

Real-World Simulation

This project mirrors actual enterprise Document Intelligence systems used in legal firms, finance, HR automation, and cybersecurity operations — where cloud AI APIs process thousands of documents daily.

2. Core Objectives

By the end of this project, students will be able to:

- Design and build multi-node automation workflows in n8n
- Authenticate and call the Google Gemini API (gemini-3-flash)
- Engineer effective prompts for structured AI extraction
- Handle multi-modal inputs: PDF text and embedded images via Gemini Vision
- Build and integrate a Python microservice (Flask/FastAPI)
- Route AI output to Google Sheets (cloud results database)
- Implement error handling, retries, and logging inside n8n
- Send real-time email notifications via Gmail when a document is processed

3. System Architecture

3.1 Components

Component	Description
n8n Workflow Engine	Local install or Docker — orchestrates all automation steps
Google Gemini API	Cloud LLM — gemini-3-flash model for text analysis & vision
Student Metadata API	Python Flask/FastAPI microservice for enrichment logic
Cloud Storage Layer	Google Sheets — used as the results database
Filesystem Trigger	Watched folder (incoming_docs/) for file drop detection
Email Notification	Gmail node sends a real-time email summary on completion
Output Folder	Processed JSON + summary reports written to output_docs/

3.2 Technology Stack

Area	Technology
AI Model	Google Gemini 3 Flash (via REST API)
Automation	n8n (self-hosted, v1.x)
Microservice Language	Python 3.10+ (Flask or FastAPI)
Cloud Storage	Google Sheets API (via the n8n Google Sheets node)
Notifications	Gmail (via the n8n Gmail node — OAuth2 authentication)

File Parsing	PyMuPDF (PDF), python-docx (DOCX), plain text
Authentication	Gemini API Key (Google AI Studio)

3.3 Architecture Diagram

The complete pipeline visualized end-to-end. Each colored layer represents a stage: input detection, file routing, text extraction, AI analysis with Gemini 3, metadata enrichment, and the three output channels (Google Sheets, output file, email).

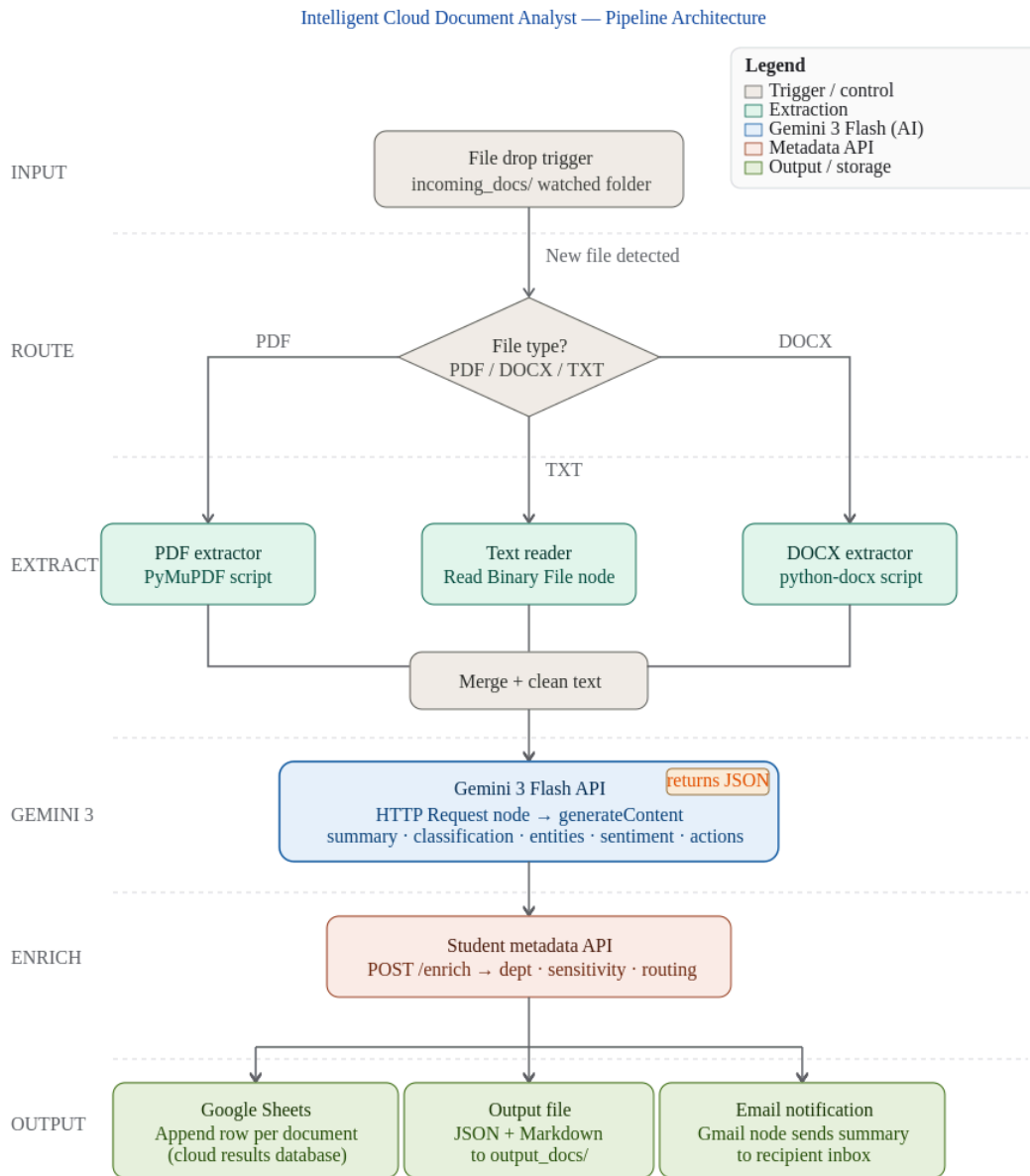


Figure 1 — End-to-end pipeline architecture

4. Functional Pipeline

The complete workflow runs in the following sequence:

1. **Step 1: File Detection** — A user drops a document into the local `incoming_docs/` folder.
2. **Step 2: Text + Image Extraction** — n8n reads and extracts text content using an Execute Command or Code node. PDF files are parsed using pdfplumber or PyMuPDF via a Python script. Images embedded in PDFs are optionally passed to Gemini Vision.
3. **Step 3: Gemini AI Analysis** — n8n sends the extracted text to the Gemini API using an HTTP Request node. The prompt requests a structured JSON response with: summary, classification, entities, sentiment, and action items.
4. **Step 4: Metadata Enrichment** — n8n sends the same content to the student-built Metadata API. The API adds enriched metadata: sensitivity level, keyword tags, department routing, and a confidence score.
5. **Step 5: Google Sheets Write** — n8n parses the JSON output from Gemini and appends a row per document into the Google Sheets results database.
6. **Step 6: Output Generation + Email Notification** — n8n compiles a final human-readable summary (JSON + Markdown) and writes it to `output_docs/`, then sends an email via the Gmail node with the summary and routing tag.

Key Upgrade: Gemini Vision

If a document contains images (charts, scanned pages, diagrams), students can optionally pass the image bytes to Gemini's vision capability to extract visual information — something not possible with text-only Ollama models.

5. Gemini API Integration

5.1 Getting an API Key

- Visit Google AI Studio: <https://aistudio.google.com/app/apikey>
- Sign in with a Google account and click 'Create API Key'
- Store the key as an n8n Credential (HTTP Header Auth: x-goog-api-key)
- Never hardcode API keys — always use n8n's credential manager

5.2 Recommended Model

Model	Use Case
gemini-3-flash	Fast, cost-efficient, excellent for document analysis (RECOMMENDED)
gemini-3-pro	Higher accuracy for complex extraction, larger context window

gemini-3-flash (vision)

Use for image/PDF visual extraction tasks

5.3 n8n HTTP Request Node Setup

Configure the n8n HTTP Request node as follows:

```
Method:  POST
URL:
https://generativelanguage.googleapis.com/v1beta/models/gemini-3-flash:generateContent

Headers:
  Content-Type:    application/json
  x-goog-api-key:  {{ $credentials.geminiApiKey }}

Body (JSON):
{
  "contents": [{
    "parts": [{ "text": "{{ $json.prompt }}" }]
  }],
  "generationConfig": {
    "temperature": 0.2,
    "responseMimeType": "application/json"
  }
}
```

5.4 Structured Output Prompt Template

Students must write a prompt that instructs Gemini to return valid JSON only. Example prompt:

```
You are a document intelligence assistant. Analyze the following document text
and return ONLY a valid JSON object with these exact fields:

{
  "summary": "2-3 sentence summary of the document",
  "classification": "one of: [invoice, report, contract, ticket, article, other]",
  "sentiment": "one of: [positive, neutral, negative]",
  "entities": {
    "people": ["list of person names"],
    "organizations": ["list of org names"],
    "dates": ["list of dates mentioned"],
    "amounts": ["list of monetary amounts"]
  },
  "action_items": ["list of recommended actions"],
  "confidence_score": 0.0 to 1.0
}
```

```
Document text:
{{ $json.extracted_text }}
```

6. Student Metadata API

Students must build a lightweight Python microservice that enriches the Gemini output with business-specific metadata. This simulates real-world API integration patterns where an internal company service adds domain knowledge to AI results.

6.1 Required Endpoints

Endpoint	Description
POST /enrich	Accepts Gemini JSON output, returns enriched metadata
GET /health	Health check endpoint — must return {"status": "ok"}
GET /categories	Returns the list of available document categories
POST /sensitivity	Classifies document sensitivity: public / internal / confidential

6.2 Enrichment Logic Requirements

- Map Gemini classification to an internal department (e.g., 'invoice' → Finance)
- Assign a sensitivity level based on keywords found in entities
- Generate routing tags (e.g., 'needs-review', 'auto-approved', 'escalate')
- Add a timestamp and a unique document ID (UUID)
- Return a confidence score adjustment based on entity completeness

```
# Minimal FastAPI example
from fastapi import FastAPI
from pydantic import BaseModel
import uuid, datetime

app = FastAPI()

class GeminiResult(BaseModel):
    classification: str
    sentiment: str
    confidence_score: float
    entities: dict
```

```
@app.post('/enrich')
def enrich(data: GeminiResult):
    dept_map = {'invoice': 'Finance', 'contract': 'Legal', 'report': 'Management'}
    return {
        'document_id': str(uuid.uuid4()),
        'department': dept_map.get(data.classification, 'General'),
        'sensitivity': 'confidential' if 'amount' in str(data.entities) else
'internal',
        'routing_tag': 'needs-review' if data.confidence_score < 0.7 else
'auto-approved',
        'processed_at': datetime.datetime.utcnow().isoformat()
    }
```

7. Google Sheets Integration

Instead of writing to a local CSV or SQLite database, students must integrate Google Sheets as the cloud results database. Each processed document appends one row to a shared spreadsheet. This teaches OAuth2 authentication and real cloud API patterns.

7.1 Setup Steps

7. Create a new Google Sheet and add the required column headers in row 1
8. In n8n: Credentials → New → Google Sheets OAuth2 API
9. Follow the n8n OAuth flow to authorize access to your Google account
10. Add a Google Sheets node to your workflow with operation 'Append'
11. Map fields from the previous nodes to the spreadsheet columns

7.2 Required Columns in Google Sheets

- document_id (UUID)
- filename and file type
- processed_at (ISO timestamp)
- classification and department
- sentiment and confidence_score
- summary (truncated to 500 chars)
- routing_tag and sensitivity
- action_items (as comma-separated text)

Why Google Sheets?

Google Sheets is free, requires no extra setup, and lets students see their results update in real time. It also makes it trivial to build dashboards using built-in charts or to share results with the instructor for grading.

8. Email Notification (Gmail)

After every successful run, n8n must send a real-time email to a configured recipient. The email contains the document summary, classification, sensitivity, and a link to the row in Google Sheets.

8.1 Gmail Node Setup

12. In n8n: Credentials → New → Gmail OAuth2 API
13. Authorize access to your Gmail account through the OAuth flow
14. Add a Gmail node to your workflow (operation: 'Send')
15. Set the 'To' field to a configurable recipient address
16. Build the email body using expressions from previous nodes

8.2 Recommended Email Template

```
Subject: [{{$json.classification}}] New document processed: {{$json.filename}}
```

Body (HTML):

```
<h2>Document Processed</h2>
<p><b>File:</b> {{$json.filename}}</p>
<p><b>Classification:</b> {{$json.classification}}</p>
<p><b>Sentiment:</b> {{$json.sentiment}}</p>
<p><b>Department:</b> {{$json.department}}</p>
<p><b>Sensitivity:</b> {{$json.sensitivity}}</p>
<p><b>Routing tag:</b> {{$json.routing_tag}}</p>
<h3>Summary</h3>
<p>{{$json.summary}}</p>
<h3>Action items</h3>
<p>{{$json.action_items}}</p>
<hr>
<p><i>Sent automatically by n8n + Gemini 3 Document Analyst</i></p>
```

9. Bonus Challenges

Students who complete the core project can tackle one or more of these bonus challenges for extra credit:

Challenge	Description
 Gemini Vision	Pass embedded PDF images to Gemini 3 Flash Vision for visual analysis (charts, scanned pages)
 Daily Email Digest	Add a scheduled n8n workflow that sends a daily summary email of all documents processed in the past 24 hours
 Live Dashboard	Build a simple HTML/JS dashboard that reads from Google Sheets and shows live stats and charts
 Retry Logic	Implement n8n error handling with automatic retry on Gemini API rate limit errors
 Semantic Search	Store Gemini embeddings in a vector DB (Supabase / Pinecone) for document search
 Multi-model Compare	Call both Gemini 3 Flash and Gemini 3 Pro and compare their extraction quality side by side
 Multi-file Batch	Extend the workflow to process a ZIP file of multiple documents in parallel
 Sensitivity Alerting	Send a high-priority email immediately when a 'confidential' document is detected

10. Scenarios Students Can Choose

Each student selects ONE domain scenario for their documents. All prompts and enrichment logic must be tailored to the chosen scenario.

17. Business documents — contracts, invoices, purchase orders, vendor agreements
18. Cybersecurity incident logs — alert reports, SIEM exports, vulnerability scans
19. Financial reports — earnings summaries, budget proposals, expense reports
20. HR and ticket summaries — employee complaints, support tickets, onboarding docs
21. Product descriptions — e-commerce listings, spec sheets, marketing copy
22. Academic articles — research abstracts, literature reviews, thesis summaries
23. Legal documents — court filings, NDAs, compliance reports (ADVANCED)
24. Medical summaries — clinical notes, discharge summaries, lab results (ADVANCED)

